

## 05-前缀和

## 05-前缀和

### 一、本阶段目标

---

前缀和是 CSP 中非常重要的基础算法之一。

这一章的目标是：

- 理解为什么普通区间求和太慢
- 理解“预处理”思想
- 掌握一维前缀和
- 掌握二维前缀和
- 能快速求任意区间和
- 理解二维公式的推导过程
- 熟悉前缀和常见下标问题

前缀和的核心思想是：

提前计算一部分信息，后面快速查询。

也就是：

用空间换时间。

---

## 二、为什么需要前缀和？

### 1. 普通区间求和

---

例如：

数组：  
1 2 3 4 5

要求：

求第 2 个到第 4 个数的和

普通做法：

```
int sum=0;

for(int i=2;i<=4;i++)
{
    sum+=a[i];
}
```

时间复杂度：

$O(n)$

## 2. 如果查询很多次怎么办？

例如：

有  $10^5$  次区间查询

如果每次都遍历：

$O(n \times q)$

会非常慢。

于是我们想到：

能不能提前把前面的和都算出来？

这就是前缀和。

## 三、一维前缀和

### 1. 前缀和定义

定义：

$s[i]$  表示前  $i$  个数的和。

例如：

```
a:
1 2 3 4 5
```

前缀和：

```
s:
1 3 6 10 15
```

因为：

```
s[1]=1
s[2]=1+2=3
s[3]=1+2+3=6
...
```

---

## 2. 前缀和递推公式

```
s[i]=s[i-1]+a[i];
```

含义：

```
前 i 个数的和
=
前 i-1 个数的和
+
第 i 个数
```

---

## 3. 为什么很多人喜欢从 1 开始存？

因为这样：

```
s[r]-s[l-1]
```

会非常自然。

如果从 0 开始：

```
边界会更容易乱。
```

所以 CSP 中前缀和通常：

数组从 1 开始。

## 4. 一维前缀和模板

```
#include<iostream>
using namespace std;

int main()
{
    int n;
    cin>>n;

    int a[100];
    int s[100]={0};

    for(int i=1;i<=n;i++)
    {
        cin>>a[i];
    }

    for(int i=1;i<=n;i++)
    {
        s[i]=s[i-1]+a[i];
    }

    for(int i=1;i<=n;i++)
    {
        cout<<s[i]<<" ";
    }

    return 0;
}
```

## 四、区间求和

### 1. 核心公式

如果：

$s[i]$  表示前  $i$  个数的和

那么：

```
[l, r] 的区间和  
=  
s[r]-s[l-1]
```

---

## 2. 为什么成立？

---

例如：

```
a:  
1 2 3 4 5
```

前缀和：

```
s:  
1 3 6 10 15
```

要求：

```
2~4 的和
```

也就是：

```
2+3+4
```

观察：

```
s[4]=1+2+3+4  
s[1]=1
```

所以：

```
s[4]-s[1]  
=  
2+3+4
```

也就是：

```
s[r]-s[l-1]
```

---

### 3. 区间求和模板

---

```
#include<iostream>
using namespace std;

int main()
{
    int n,q;
    cin>>n>>q;

    int a[100];
    int s[100]={0};

    for(int i=1;i<=n;i++)
    {
        cin>>a[i];
    }

    for(int i=1;i<=n;i++)
    {
        s[i]=s[i-1]+a[i];
    }

    while(q-->0)
    {
        int l,r;
        cin>>l>>r;

        cout<<s[r]-s[l-1]<<"\n";
    }

    return 0;
}
```

---

## 五、前缀和真正优化了什么？

普通求区间和：

每次查询都重新加

复杂度：

$O(n)$

---

使用前缀和：

提前一次性预处理  
后面每次查询  $O(1)$

---

复杂度：

预处理： $O(n)$   
查询： $O(1)$

---

## 核心思想

前缀和真正优化的是：

重复计算。

---

## 六、二维前缀和

---

### 1. 为什么需要二维前缀和？

如果题目变成：

求矩阵某个区域的元素和

例如：

1 2 3  
4 5 6  
7 8 9

要求：

左上角到右下角某个矩形区域的和

如果每次都暴力遍历：

会很慢。

于是出现：

二维前缀和。

## 七、二维前缀和定义

定义：

$s[i][j]$

表示：

从  $(1,1)$  到  $(i,j)$  这个矩形区域的和。

例如：

1 2  
3 4

二维前缀和：

1 3  
4 10

因为：

$s[2][2]$   
=  
 $1+2+3+4$   
=  
10

## 八、二维前缀和公式

### 1. 核心公式

$s[i][j]$   
=  
 $s[i-1][j]$



```
+s[i][j-1]
-s[i-1][j-1]
+a[i][j]
```

## 2. 为什么要减？

观察：

```
s[i-1][j]
```

和：

```
s[i][j-1]
```

会有一块区域被重复计算。

也就是：

```
左上角区域
```

所以要减去：

```
s[i-1][j-1]
```

## 3. 图形理解

```
左边矩形
+
上边矩形
=
左上角被算了两次
```

所以：

```
减掉一次左上角
```

最后再加：

```
当前格子 a[i][j]
```

---

## 九、二维前缀和模板

```
#include<iostream>
using namespace std;

int main()
{
    int n,m;
    cin>>n>>m;

    int a[105][105];
    int s[105][105]={0};

    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=m;j++)
        {
            cin>>a[i][j];
        }
    }

    for(int i=1;i<=n;i++)
    {
        for(int j=1;j<=m;j++)
        {
            s[i][j]
            =
            s[i-1][j]
            +s[i][j-1]
            -s[i-1][j-1]
            +a[i][j];
        }
    }

    return 0;
}
```

---

## 十、二维区间求和

### 1. 目标

---

求矩形：

(x1,y1)  
到  
(x2,y2)

的区域和。

---

## 2. 核心公式

```
sum
=
s[x2][y2]
-s[x1-1][y2]
-s[x2][y1-1]
+s[x1-1][y1-1]
```

---

## 3. 为什么这样算？

可以理解成：

```
大矩形
-
上面不要的
-
左边不要的
+
左上角被减重了，所以补回来
```

---

## 十一、二维区间求和模板

```
#include<iostream>
using namespace std;

int main()
{
    int n,m,q;
    cin>>n>>m>>q;

    int a[105][105];
    int s[105][105]={0};
```

```

for(int i=1;i<=n;i++)
{
    for(int j=1;j<=m;j++)
    {
        cin>>a[i][j];
    }
}

for(int i=1;i<=n;i++)
{
    for(int j=1;j<=m;j++)
    {
        s[i][j]
        =
        s[i-1][j]
        +s[i][j-1]
        -s[i-1][j-1]
        +a[i][j];
    }
}

while(q--)
{
    int x1,y1,x2,y2;
    cin>>x1>>y1>>x2>>y2;

    int ans=
    s[x2][y2]
    -s[x1-1][y2]
    -s[x2][y1-1]
    +s[x1-1][y1-1];

    cout<<ans<<"\n";
}

return 0;
}

```

## 十二、为什么二维前缀和总容易错？

因为：

减法太多  
下标太多  
边界太多

尤其容易错：

$x1-1$   
 $y1-1$

---

## 建议

不要死背。

一定要理解：

大矩形  
-  
上面  
-  
左边  
+  
左上角补回来

---

## 十三、前缀和与差分的关系

前缀和：

快速查询

差分：

快速修改

两者其实是互逆关系。

后面差分专题会详细讲。

---

## 十四、前缀和的本质

前缀和真正的核心思想是：

提前预处理。

也就是：

把未来会重复计算的东西提前算好。

## 典型思维

暴力：

每次重新算

前缀和：

提前算一次，以后直接查

## 十五、常见错误

### 1. 忘记初始化 `s[0]`

正确：

```
int s[100]={0};
```

### 2. 下标从 0 开始导致边界乱

建议：

前缀和从 1 开始存。

### 3. 区间公式写错

正确：

```
s[r]-s[l-1]
```

不是：

```
s[r]-s[l]
```

## 4. 二维公式减错

正确：

```
+s[x1-1][y1-1]
```

很多人会忘记补回来。

## 5. 二维前缀和递推漏减

错误：

```
s[i][j]
=
s[i-1][j]
+s[i][j-1]
+a[i][j]
```

这样左上角会重复计算。

必须：

```
-s[i-1][j-1]
```

# 十六、本专题做过的题

| 题号    | 题目        | 类型     | 题面简述           | 对应知识点         |
|-------|-----------|--------|----------------|---------------|
| P8218 | 求区间和      | 一维前缀和  | 多次询问数组区间和      | $s[r]-s[l-1]$ |
| P2004 | 领地选择      | 二维前缀和  | 找和最大的矩形区域      | 二维区域求和        |
| P2280 | 激光炸弹      | 二维前缀和  | 求某个固定大小区域最大权值和 | 二维前缀和         |
| P2367 | 语文成绩      | 前缀思想   | 区间修改后求结果       | 前缀 / 差分前置     |
| 消消乐类题 | 模拟 / 前缀思想 | 检查连续区域 | 区域处理思想         |               |

## 十七、本章最重要的结论

前缀和的核心不是公式：

```
s[r]-s[l-1]
```

真正重要的是：

提前预处理，避免重复计算。

一维前缀和本质：

前  $i$  个数的和

二维前缀和本质：

左上角矩形的和

以后看到：

大量区间查询  
大量矩形查询

就应该想到：

前缀和。